EXERCISE 2 · DATA HANDLING WITH THE TIDYVERSE

Exercise 2 in R on your Mac

The practical recipe: what to install, what to click, what to paste where, what you will see, and what to do with it. Written for RStudio on a Mac, with your folder `Documents → ETH → HS26 → Experimental design → Ex2_folders`.

CHECK **The short version**

- You hand in **one file:** `Ex2_R_code.R`, with your answers typed into it. **No Word document**, nothing separate.
- **Code answers** (Q2, Q3, Q5, Q8) are normal R code. **Written answers** (Q1, Q4, Q6, Q7) are lines that start with `#`. R ignores those lines, so the script still runs from top to bottom.
- You add **10 things** to the script: 8 answers, 1 required fix and 1 optional extra (see the table). Everything is in section 4, ready to paste.
- Your exercise folder is `~/Documents/ETH/HS26/Experimental design/Ex2_folders` (`~` means your home folder, `/Users/yourname`). The script you edit is in its `Scripts` subfolder.
- Time: about 30 minutes to install everything, then about 1 hour for the exercise.

#	What you add	Find this text with Cmd + F, paste below it	Kind
1	Q1 answer	`#Add answer Q 1 here`	words (#)
2	ungraded answer (optional)	`select(method.id, method.label)`	words (#)
3	Q2 answer	`#Add answer Q 2 here`	code + words
4	Q3 answer	`#Add answer Q 3 here`	code
5	**required fix** (a missing file)	`"dat.long.rds"))`	code (1 line)
6	Q4 answer	`#Add answer Q 4 here`	words (#)
7	Q5 answer (2 points)	`#Add the code for Q 5 here`, **1st** match	code
8	Q6 answer	`#Add the code for Q 5 here`, **2nd** match (yes, it says Q 5)	words (#)
9	Q7 answer	`#Type your answer here.`	words (#)
10	Q8 answer	`#Type your code here` (the very last line)	code

Work through section 4 from top to bottom. **All answer code only uses things that are already in the script**; each code answer has a green line showing which part of the script it copies. The written answers are complete and correct; you may want to rephrase them a little in your own words.

1. Install everything (once)

1. **Check your folder.** In Finder, open *Documents → ETH → HS26 → Experimental design → Ex2_folders*. It must contain the three folders `Scripts`, `data` and `out`. (A folder called `__MACOSX` can be ignored.)

2. **Check your chip.** Click the Apple icon (top left of the screen) → *About This Mac*. "Chip: Apple M1/M2/M3/M4..." means Apple silicon; "Processor: ... Intel ..." means Intel.
3. **Install R** from cran.r-project.org → *Download R for macOS*. Download the `.pkg` file marked **arm64** (Apple silicon) or **x86_64** (Intel). Double-click it in your Downloads folder and click *Continue* → *Agree* → *Install*; enter your Mac password when asked.
4. **Install RStudio Desktop** (free) from posit.co/download/rstudio-desktop → the macOS download (a `.dmg` file). Open it and drag **RStudio** into the **Applications** folder.
5. **Start RStudio** (Launchpad or *Applications* → *RStudio*; not "R": from now on you always use RStudio). If macOS asks *"RStudio is an app downloaded from the Internet. Are you sure you want to open it?"*, click **Open**.
6. **Install the two packages.** Click into the **Console** (the bottom-left pane, where the `>` sign is). Type the line below, press **Enter**, and wait 2 to 10 minutes. Lots of scrolling text is normal.

```
install.packages("tidyverse")
```

Then do the same with:

```
install.packages("svglite")
```

If your Mac or RStudio asks...	answer
"Do you want to install from sources the packages which need compilation? (Yes/no/cancel)"	type `no`, press Enter
"Would you like to use a personal library instead?"	type `yes`, press Enter
to choose a "CRAN mirror"	pick any, e.g. Switzerland or "0-Cloud"
"RStudio" would like to access files in your "Documents" folder	click **Allow** / **OK**

7. Test it: type `library(tidyverse)` in the console and press Enter. You should see a block starting with `── Attaching core tidyverse packages ──` and **no** line starting with `Error`. Done: you never have to install these again.

2. Every time: open the script and set the folder

Do these two steps **every time you start RStudio** to work on the exercise:

1. *File* → *Open File...* (or **Cmd + O**). In the window, click **Documents** in the left sidebar, then double-click *ETH* → *HS26* → *Experimental design* → *Ex2_folders* → *Scripts* and open `Ex2_R_code.R`. It opens in the top-left pane.
2. In the menu bar at the top of the screen: *Session* → *Set Working Directory* → *To Source File Location*.

THE CONSOLE THEN SHOWS

```
> setwd("~/Documents/ETH/HS26/Experimental design/Ex2_folders/Scripts")
```

Alternative for step 2: if the menu item is greyed out, click into the **console** and paste the line below. It is the `setwd(...)` line from the top of the script (line 5, `#setwd("C:/Users/YOUR_PATH")`), filled in

with your folder:

```
setwd("~/Documents/ETH/HS26/Experimental design/Ex2_folders/Scripts")
```

Type it into the **console only**, not into the script: on JupyterHub that folder does not exist, so the handed-in script would break.

TRAP **Step 2 is the most common reason the exercise "does not work"**

The script finds its data with paths like `"../data/"`, which means "from the `Scripts` folder, go up one folder, then into `data`". That only works if R is "standing" in the `Scripts` folder. If you get `does not exist in current working directory`, repeat step 2.

TIP **The file may show an empty line between most lines**

That is only how the file was saved; it changes nothing. It also means the line numbers in RStudio are about twice as high as you might expect, which is why this guide always says **"search for this text"** instead of giving line numbers.

3. RStudio in two minutes

① Script (top left)
Your file `Ex2_R_code.R`. You paste your answers here. Nothing runs until you press Cmd + Enter.

Environment (top right)
Lists the tables and objects R has in memory. You can ignore it.

② Console (bottom left)
Where the results (output) appear. The `>` sign means "ready".

③ Files · Plots (bottom right)
Plots: your graphs (arrows to flip through, *Zoom* to enlarge). *Files*: browse folders.

To…	do this
run one line	click anywhere in the line, press **Cmd + Enter**. If the command spans several lines, RStudio runs all of them. The cursor then jumps to the next command, so you can keep pressing Cmd + Enter.
run several lines	select them with the mouse, press **Cmd + Enter**
find a placeholder	**Cmd + F** opens a search bar above the script. Type the text, press Enter (again for the next match), then **Esc**.
add an answer	click at the **end** of the placeholder line (or press **Cmd + →**), press **Enter** for a new line, paste with **Cmd + V**
save	**Cmd + S**. The file name in the tab turns from red to black.
start over	*Session → Restart R*, set the working directory again (section 2), then run the script from the top
get out of a stuck console	if the console shows `+` instead of `>`, click into it and press **Esc**

To…	do this
type `|` `[ ]` `{ }` `~`	Swiss/German Mac keyboard: **Option + 7**; **Option + 5 / 6**; **Option + 8 / 9**; **Option + N**, then Space

Red text in the console is only a problem if the line starts with `Error`. Everything else (messages, `Rows: 36 Columns: 6`, the tidyverse start-up text) is just information.

> **TIP** **Copying from this PDF**
>
> You can copy the grey code boxes and paste them into RStudio. After pasting, check that the line breaks look exactly like in the PDF: every line that starts with `#` must be on its own line. If a `#` line got glued to the front of a code line, R would ignore that code. If quotes turn curly (`“ ”`), retype them as straight quotes `"`.

4. The exercise, round by round

Go through the script from top to bottom. A grey **round** means: just run the template code that is already there. A purple card means: **paste an answer** at that place, then run it. Never skip a round: later lines need the results of earlier ones.

1 Start-up

RUN Select from `#Clean your workspace` down to `library(tidyverse)`, press **Cmd + Enter**.

YOU SEE `dir()` prints `"Ex2_R_code.R"`. `library(tidyverse)` prints the start-up block (a "message", not an error).

DO Check `dir()`: if it shows `"data" "out" "Scripts"` instead, the working directory is wrong; redo section 2.

2 Read the measurement table

RUN Select from `getwd()` down to `head(measurement)`, press **Cmd + Enter**.

YOU SEE `"/Users/yourname/Documents/ETH/HS26/Experimental design/Ex2_folders/Scripts"`, then `Rows: 36 Columns: 6` and a table of 6 rows:

```
# A tibble: 6 × 6
  measurement.id plot.id trait.id method.id timestamp  value
           <dbl>   <dbl>    <dbl>     <dbl> <chr>      <dbl>
1              1       1        1         1 2014.04.14  48.2
2              2       2        1         1 2014.04.14  49
…
```

DO nothing. Continue with Q1.

Q1 · 1 point · written answer

1. Press **Cmd + F**, search for `#Add answer Q 1 here`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
# Q1: measurement.csv has 3 comment lines at the top (the ones with #).
# They say who made the file, what it's for and what the data is
# (data from an ETH plant nutrition excursion, April 2014).
# Comment lines can be used to write info about the data directly into
# the file, like author, date, units or notes. read_csv ignores them
# because of comment="#".
```

3. Nothing to run: lines starting with `#` do nothing.

   Where the "3" comes from: in Finder, right-click `Ex2_folders/data/measurement.csv` → *Open With* → *TextEdit*, and count the lines starting with `#` at the top.

3 Pipe examples

RUN	Select from `1:10 |> mean()` down to `years |> as.character() |> as.numeric()`, press **Cmd + Enter**.
YOU SEE	`5.5`, `5.5`, then `2008 2009 2010 2011 2012` twice.
DO	nothing.

4 Read the other tables

RUN	Select from `design <- read_csv(` down to `select(method.id, method.label)`, press **Cmd + Enter**.
YOU SEE	four short `Rows: … Columns: …` messages.
DO	nothing (optionally add the ungraded answer below).

Ungraded question · optional · written answer

1. Press **Cmd + F**, search for `select(method.id, method.label)`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
# Ungraded: the SPAD meter (e.g. Minolta SPAD-502) measures leaf chlorophyll
# without damaging the leaf. It measures how much red (650 nm) and
# infrared (940 nm) light goes through the leaf and gives a value
# from 0 to 99.9. It's a relative index, not the real chlorophyll content.
# Normally it's measured on the flag leaf.
```

3. Nothing to run. (There is no placeholder for this question in the script, so this is a sensible place for it. You may also skip it: it gives 0 points.)

5 Check the keys of the entry table

RUN	Select from `entry |>` down to `filter(n > 1)`, press **Cmd + Enter**.
YOU SEE	first a table with `Tapidor 3` and `Titlis 3`, then `# A tibble: 0 × 2`. (There are two `entry |>` blocks; run both.)
DO	nothing.

Q2 · 1 point · code + written answer

1. Press **Cmd + F**, search for `#Add answer Q 2 here`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
design |>
  count(plot.id) |>
  filter(n > 1)
trait |>
  count(trait.id) |>
  filter(n > 1)
# Q2: Both checks give a tibble with 0 rows, so no id shows up twice.
# plot.id (design) and trait.id (trait) are unique.
```

Built from the script: the block `entry |> count(entry.id) |> filter(n > 1)` right above, once with `design` and `plot.id`, once with `trait` and `trait.id`.

3. Select the 6 code lines you pasted (from `design |>` to the second `filter(n > 1)`) and press **Cmd + Enter**.
4. You see twice:

```
# A tibble: 0 × 2
# ℹ 2 variables: plot.id <dbl>, n <int>
# A tibble: 0 × 2
# ℹ 2 variables: trait.id <dbl>, n <int>
```

What it means: 0 rows = no duplicates = the key is unique. That is exactly what the two `#` lines of your answer say, so there is nothing more to do.

6 Join the tables

RUN	Select from `#Do a left join` down to `left_join(trait, by="trait.id")`, press **Cmd + Enter**.
YOU SEE	nothing. Assigning with `<-` is silent.
DO	nothing. Continue with Q3.

Q3 · 1 point · code

1. Press **Cmd + F**, search for `#Add answer Q 3 here`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
dat.long <- dat.long |>
  left_join(method, by="method.id")
```

   Built from the script: the join right above: one more `left_join(…)` like `left_join(trait, by="trait.id")`, with `method` and `"method.id"`.

3. Click into the first pasted line and press **Cmd + Enter** (both lines run).
4. You see nothing. The check comes in the next round: the second `str(dat.long)` shows `tibble [36 × 15]` and its last line is `$ method.label`.

   Paste Q3 **exactly here**, before the next block. If you put it further down, you get the error `Can't join … due to incompatible types`.

7 Turn columns into factors

RUN	Select from `#Identify all ids` down to `str(dat.long)` (the second one), press **Cmd + Enter**.
YOU SEE	two long `str()` listings. In the second, most columns say `Factor`, and the last one is `$ method.label` (thanks to Q3).
DO	nothing.

8 Dates

RUN	Select from `dat.long$timestamp <-` down to `str(dat.long$DOY)`, press **Cmd + Enter**.
YOU SEE	many `15` s (week), many `2` s (quarter), then `num [1:36] 104 105 104 105 …`.
DO	nothing. Later the dates show as 2014-04-13 and 2014-04-14, one day earlier than in the file. That is a time-zone quirk and is expected (the tutorial shows the same).

9 Treatment label and save

RUN	Select from `dat.long <- dat.long |> mutate(treat` down to `write_rds(dat.long, file=paste0(outputPath, "dat.long.rds"))`, press **Cmd + Enter**.
YOU SEE	nothing.
DO	add the fix below **right now**.

Required fix · 0 points, but without it the script crashes later

1. Press **Cmd + F**, search for `"dat.long.rds"))`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
write_rds(dat.long, file=paste0(outputPath, "dat.longFull.rds"))
```

   Built from the script: the `write_rds` line right above, only with the file name `"dat.longFull.rds"` (the name the script loads later).

3. Click into the line and press **Cmd + Enter**.
4. You see nothing. In the *Files* pane, open the `out` folder: it now contains `dat.long.rds` and `dat.longFull.rds`.

   Why: further down, the script loads `../out/dat.longFull.rds`, a file that is missing from your zip. Without this line you get `cannot open file '../out/dat.longFull.rds': No such file or directory`.

10 Field plots

RUN	Select from `#Only the day of the year (DOY) 104` down to `plot(p3)`, press **Cmd + Enter**.
YOU SEE	three plots in the *Plots* pane (flip with the arrows): an empty grid, coloured tiles, and tiles with labels such as `0.Tapidor`.
DO	nothing. They look slightly different from the pictures in the tutorial (axis labels 1, 2, 3 instead of 1.0, 1.5, ...). That is fine.

11 Both days side by side

RUN	Select from `#Both days together` down to `plot(p_facet)`, press **Cmd + Enter**.
YOU SEE	the same field twice, side by side, with panels labelled 104 and 105.
DO	nothing.

12 Save the graph as PNG and SVG

RUN	Select from `p<-ggplot(dat.long, aes(row, range))` down to `ggsave(p, filename = paste0(outputPath, "rastergraph.svg"), device="svg")`, press **Cmd + Enter**.
YOU SEE	`Saving 7 x 7 in image` twice (your numbers may differ). The `out` folder now contains `rastergraph.png` and `rastergraph.svg`.
DO	if you get `there is no package called 'svglite'`, run `install.packages("svglite")` in the console and run the `ggsave` lines again.

Q4 · 1 point · written answer

1. Press **Cmd + F**, search for `#Add answer Q 4 here`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
# Q4: When I zoom in, the png gets pixelated and blurry, the svg stays sharp.
# The png is a raster image, so it's made of a fixed number of pixels.
# The svg is a vector image, it stores the shapes and text as instructions,
# so it can be scaled to any size and the parts (text, colours...) can
# still be edited later, e.g. in Inkscape or Illustrator.
# For further processing or scaling I would take the svg.
```

3. Nothing to run.

   To see it with your own eyes (optional): in Finder, open `Ex2_folders/out`, drag both files into a Safari or Chrome window and zoom in a lot (**Cmd + plus**, or pinch on the trackpad). The PNG gets blocky, the SVG stays sharp.

13 Line plot (prepared, but not shown)

RUN	Select from `#Prepare the panel` down to `p<-p + theme(axis.text.x = element_text(angle = 90))`, press **Cmd + Enter**.
YOU SEE	nothing: this block only builds the plot `p`.
DO	nothing (to see it anyway, run `plot(p)`). Continue with Q5.

Q5 · 2 points · code

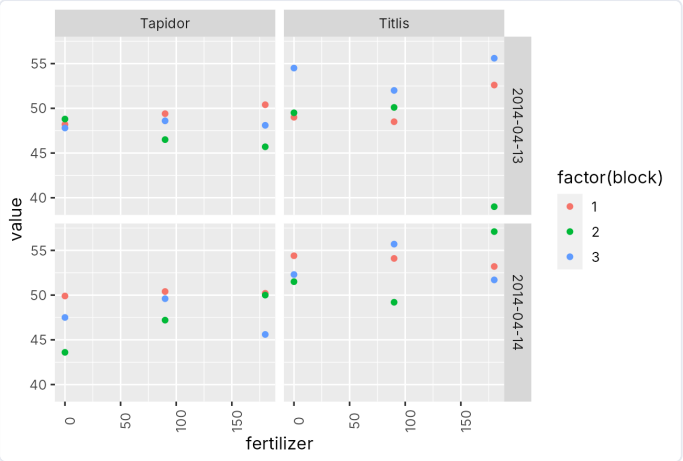
1. Press **Cmd + F**, search for `#Add the code for Q 5 here` (the **1st** match: press Enter in the search box until you are there). Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
p <- ggplot(dat.long |> mutate(fertilizer = as.numeric(as.character(fertilizer))),
            aes(x=fertilizer, y=value))
p <- p + geom_point(aes(x=fertilizer, y=value, colour=factor(block)))
p <- p + scale_color_discrete()
p <- p + facet_grid(date ~ variety)
p <- p + theme(axis.text.x = element_text(angle = 90))
plot(p)
```

   Built from the script: the line-plot block right above (`p <- ggplot(…)`, then `p <- p + …`), with `geom_point` instead of `geom_line` and `facet_grid(date ~ variety)`. `dat.long |> mutate(…)` inside `ggplot(…)` works like `dat.long |> filter(DOY == 104)` in `p1`, and `mutate` is the same function the script uses for `treat`. `as.numeric(as.character(…))` is the `years` example near the top of the script: it turns the fertilizer factor back into the numbers 0, 90, 180, which gives the number axis of the target plot.

3. Select all 7 pasted lines and press **Cmd + Enter**.
4. The plot appears in the *Plots* pane. Compare it with the target below: points, fertilizer 0 to 180 on the x-axis (ticks 0, 50, 100, 150), Tapidor and Titlis as columns, the two dates as rows, colours for blocks 1 to 3.

   Do not change `dat.long <- …` here. The conversion of fertilizer to numbers happens only inside this plot, which is intended: the ANOVA later needs the original version.



What Q5 should look like. Click *Zoom* in the Plots pane to see it bigger.

14 Make the wide table

RUN Select from `dat.long<-read_rds(file=paste0(outputPath, "dat.longFull.rds"))` down to `head(dat.wide)`, press **Cmd + Enter**.

YOU SEE a table of 6 rows; the last lines mention the two new columns:

```
# A tibble: 6 × 13
  plot.id entry.id block row   range variety fertilizer trait.id method.id
…
# ℹ 4 more variables: trait.name <chr>, treat <chr>,
#   `Fl0Chl_SPAD001_2014-04-13` <dbl>, `Fl0Chl_SPAD001_2014-04-14` <dbl>
```

DO nothing. Continue with Q6 (right below `head(dat.wide)`).

Q6 · 1 point · written answer

1. Press **Cmd + F**, search for `#Add the code for Q 5 here` (the **2nd** match: press Enter in the search box until you are there). Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
# Q6: The values from trait.label, method.label and date were put together
# with "_" and are now the names of the new columns:
# Fl0Chl_SPAD001_2014-04-13 and Fl0Chl_SPAD001_2014-04-14.
# The SPAD values went into these two columns, so the three columns
# (and value) are gone and each plot has just one row with both days
# next to each other. That's why there are 18 rows now instead of 36.
```

3. Nothing to run.

   Yes, this placeholder says "Q 5" a second time. It is a mistake in the template; this one is for Q6.

15 Compare the two days

RUN Select from `TraitsWide<-dat.wide` down to `plot(TraitsWide)`, press **Cmd + Enter**.

YOU SEE a simple scatter plot with 18 circles (day 1 on x, day 2 on y).

DO nothing.

16 Split the data by date

RUN — Select from `test<-dat.long |> split(dat.long$date)` down to `str(test)`, press **Cmd + Enter**.

YOU SEE — a long listing that starts like this:

```
List of 2
 $ 2014-04-13: tibble [18 × 18] (S3: tbl_df/tbl/data.frame)
  ..$ plot.id       : Factor w/ 18 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 9 10
...
  …
 $ 2014-04-14: tibble [18 × 18] (S3: tbl_df/tbl/data.frame)
  …
```

DO — nothing. Continue with Q7.

Q7 · 1 point · written answer

1. Press **Cmd + F**, search for `#Type your answer here.`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
# Q7: split() returns a list with 2 elements, one for each date
# (2014-04-13 and 2014-04-14). Each element is a tibble with the
# 18 plots measured on that day and all 18 columns (tibble [18 x 18]).
# So the data is just divided into two parts, nothing gets lost.
```

3. Nothing to run.

17 One ANOVA per day

RUN — Select from `aov1<-dat.long |> split(dat.long$date) |>` (the first one) down to `Pr.val`, press **Cmd + Enter**.

YOU SEE — two ANOVA printouts, a long `str()` listing, and finally the p-values:

```
$`2014-04-13`
[1] 0.2907863 0.8806472 0.1430874 0.9188675        NA

$`2014-04-14`
[1] 0.001973723 0.604497550 0.328610606 0.814910513          NA
```

DO — nothing. Continue with Q8.

Q8 · 1 point · code

1. Press **Cmd + F**, search for `#Type your code here`. Press **Esc**, then click at the **end** of that line and press **Enter** to get a new empty line.
2. Paste (or type) exactly this:

```
F.val <- aov.tab |> map(function(x){x[[1]]$`F value`})
F.val
```

Built from the script: the `Pr.val` line right above, with `F value` instead of `Pr(>F)` (the column name as printed in the ANOVA table).

3. Select both pasted lines and press **Cmd + Enter**.
4. You see:

```
$`2014-04-13`
[1] 1.24400422 0.12872736 2.37648907 0.08533331         NA

$`2014-04-14`
[1] 17.2390982  0.5295666  1.2464561  0.2089240         NA
```

What it means: these are the F values of variety, fertilizer, block, variety:fertilizer and the residuals for each day. The `NA` at the end is normal (residuals have no F value). Nothing more to write: Q8 only asks for the code.

5. Final check and hand-in

1. **Save** with Cmd + S.
2. **Test the whole script in one go:** *Session → Restart R*. Then *Session → Set Working Directory → To Source File Location*. Then click the **Source** button (top right of the script pane). R runs the whole file. If it stops with a red `Error`, see section 6. If it finishes without `Error`, you are done.
3. **Check the `out` folder** (Files pane): `dat.long.rds`, `dat.longFull.rds`, `rastergraph.png`, `rastergraph.svg`.
4. **Hand in the file** `Ex2_R_code.R` from *Documents → ETH → HS26 → Experimental design → Ex2_folders → Scripts*. The course says: *"Submit your exercise by selecting the file in the repository for exercise 2"* on JupyterHub. So log in to JupyterHub (via Moodle), open the exercise 2 folder, click the **Upload** button (↑) in the file browser, pick your finished `Ex2_R_code.R` from the folder above (replace the template if it asks), and submit it there.

TIP **Not sure which button submits?**

The tutorial itself says the submission process on JupyterHub is still being worked out. If it is not clear on Moodle, ask the teaching assistant: "Do I submit the .R file in the exercise 2 folder on JupyterHub?" Do not rename the file.

- [ ] All 8 answers (plus the fix) are in the script, each right below its placeholder.
- [ ] Every line of a written answer starts with `#`.
- [ ] Source runs the whole script without any `Error`.
- [ ] Saved, uploaded to the exercise 2 folder on JupyterHub, and submitted. No Word file needed.

6. If something goes wrong

You see…	Do this
`does not exist in current working directory`	Session → Set Working Directory → To Source File Location (or the `setwd(...)` line from section 2 in the console), then run that part again.
`cannot change working directory`	A typo in the `setwd(...)` path. Copy it again from section 2, or use the menu instead.
Files "do not exist" although Finder shows them with a small cloud icon	Your Documents folder is stored in iCloud. In Finder, right-click `Ex2_folders` → *Download Now*, then try again.
`could not find function "read_csv"` (or `"ggplot"`, `"week"`, `"map"`)	Run `library(tidyverse)`. You probably restarted RStudio and skipped round 1.
`there is no package called 'tidyverse'` (or `'svglite'`)	Run `install.packages("tidyverse")` (or `"svglite"`) once in the console.
`object 'dat.long' not found` (or any other object)	You skipped a round. Run the script from the top again (or use the Source button).
`cannot open file '../out/dat.longFull.rds'`	The required fix (after round 9) is missing or was not run. Add or run it.
`Can't join x$method.id with y$method.id due to incompatible types`	Q3 is in the wrong place. Move it directly below `#Add answer Q 3 here`, restart R, run from the top.
`Column method.label doesn't exist`	Q3 is missing or was not run. Add it, restart R, run from the top.
`unexpected symbol` / `unexpected ')'` / `unexpected input`	Something got mangled while pasting. Compare the line with this PDF (quotes, brackets, line breaks).
The console shows `+` and nothing happens	Click into the console, press Esc, then check the line for a missing `)` or `"`.
Q8 shows other numbers, or the ANOVA shows Df `1` for fertilizer	You changed `dat.long` somewhere (e.g. in Q5). Use the Q5 code exactly as given, restart R, run from the top.
Weird behaviour you cannot explain	Session → Restart R, set the working directory, click Source.

Every piece of code and output in this guide was tested by running the complete, finished script (R 4.3.3, tidyverse 2.0.0) on the files from Ex2_folders.zip.